

TEMPO: Learning Temporal Context for Dynamic Robot Manipulation

Zhenyang Feng* Jimin Heo* Erik B. Sudderth Unnat Jain

University of California, Irvine

{dfeng8, heoj4, sudderth, unnatj}@uci.edu

Abstract:

Vision-language-action (VLA) models have achieved impressive performance in quasi-static manipulation, but struggle in dynamic manipulation tasks because they operate on a single observation at inference time. We identify two representational failures that underlie this limitation. The first is *motion ambiguity*, where a single observation does not include scene dynamics and therefore cannot anticipate the future state of moving objects. The second is *state aliasing*, where visually similar observations from different points in a task require different actions. We argue that these failures persist regardless of model scale and inference latency, showing that the bottleneck is missing temporal context rather than model capacity. Based on this insight, we propose TEMPO, which augments a pretrained VLA with two temporal inputs: a motion summary extracted from a frozen video foundation model to resolve motion ambiguity and a compact proprioceptive history to resolve state aliasing. TEMPO requires no modification to the backbone and adds minimal compute overhead at training or deployment. Across four dynamic manipulation tasks, it improves Bottle Handover success from 44% to 74% and is the only method that solves state aliasing. Probing and ablation studies confirm that each temporal signal independently addresses its corresponding failure. We further release TEMPO-Bench, a benchmark of over 50k annotated frames for evaluating motion-aware robot perception in both regression and multiple-choice formats. Project Website: <https://tempo-robot.github.io/>

Keywords: VLA Models; Dynamic manipulation; Representation Learning

1 Introduction

Vision-language-action models have demonstrated strong generalization on quasi-static manipulation, where objects remain at rest throughout the inference interval and a single-frame observation provides a sufficient basis for action [1, 2, 3]. This premise does not hold when objects move. A robot intercepting a bottle from a person walking past must anticipate where the bottle will be when the grasp executes, not merely where it was observed at inference time. A single frame samples the scene’s configuration but contains no information about how it is evolving. Reducing inference latency narrows but does not close this gap. Even with the freshest possible observation, a snapshot encodes the configuration of the scene, not motion (Fig. 1). We refer to this failure mode as *motion ambiguity*.

A distinct failure mode arises from task structure. Tasks that unfold over sequential subtasks produce perceptually similar frames at qualitatively different points: a hand approaching at the start of a grasp and the same hand receding at its conclusion can appear nearly identical, yet require opposite actions. A policy conditioned only on the current frame cannot disambiguate them, producing jitter and mistimed commitments even on tasks it can nominally complete. We refer to this failure mode as *state aliasing*. Both failures follow from the Markovian assumption shared by every pretrained VLA today. Section 3 shows each empirically on current baselines and identifies the temporal context each requires.

*Equal contribution.

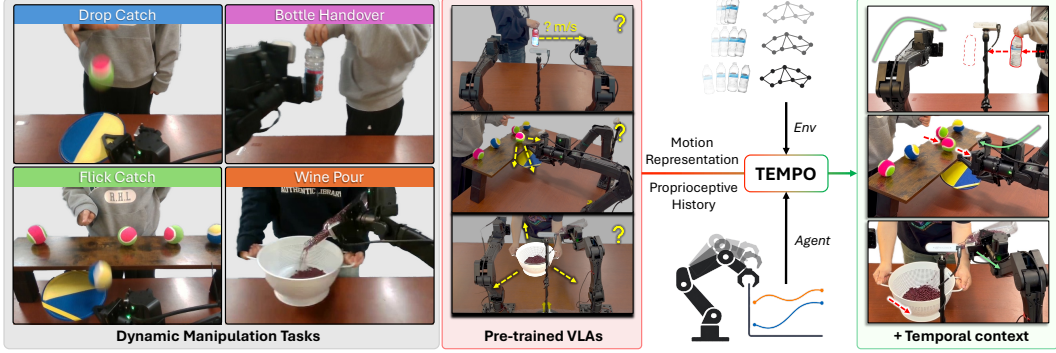


Figure 1: **TEMPO makes a pretrained VLA motion-aware.** Four dynamic tasks (*left*) in which the target object is in motion throughout the episode. A standard VLA, conditioned on a single observation, cannot determine how the scene is evolving (*center, red*; “? m/s”) and cannot disambiguate perceptually identical observations that arise at different phases of the same task, producing failed grasps and mistimed actions. TEMPO resolves both failure modes by augmenting the policy with two compact signals: TEMPO_{MOT}, a motion representation from a frozen video foundation model, and TEMPO_{ACT}, a compact proprioceptive history providing subtask context. Together they enable reliable task completion across all four tasks (*right, green*).

We present TEMPO (Fig. 1), which addresses both failure modes via two compact temporal signals that build on representations already in wide use, without modifying the pretrained backbone. The first, TEMPO_{MOT}, is a motion representation derived from a frozen video foundation model. Such models, trained on dense visual correspondence, separate scene dynamics from appearance without task-specific supervision, providing the temporal information missing from a single frame. The second, TEMPO_{ACT}, is a compact summary of the robot’s own recent proprioceptive history, providing a task-agnostic cue for which subtask is in progress and distinguishing observations that would otherwise appear identical at different task phases. Both signals require no per-task engineering and together add only 2M parameters (0.08% overhead), leaving the pretrained backbone entirely intact. Asynchronous inference [4, 5] addresses a complementary bottleneck of execution latency; we build TEMPO on top of an asynchronous backbone [5] so the gain over the async-only baseline isolates the representational contribution.

We evaluate on a bimanual platform across four dynamic tasks designed to expose each failure mode in isolation and combination: Bottle Handover, Drop Catch, Flick Catch, and Wine Pour. On tasks where motion perception or state disambiguation is the limiting factor, TEMPO outperforms both asynchronous baselines: Bottle Handover improves from 44% to 74%, and TEMPO is the only system to complete the full versions of Wine Pour and Flick Catch, where both baselines score 0% due to unresolved state aliasing. Results on Drop Catch further validate the framework: it is the one task where execution timing dominates over representation, and it is precisely where asynchronous inference shines, consistent with the complementary relationship we describe above (Fig. 2). Ablations and probing analyses confirm that each signal is individually necessary, relied upon at distinct moments (TEMPO_{MOT} during tracking, TEMPO_{ACT} at commitment), and that the motion token encodes object velocity.

We argue that extending pretrained VLAs to dynamic settings is primarily a question of representation, not scale: the temporal context needed to act in a moving scene decomposes into two distinct signals, each tied to a distinct failure mode and independently verifiable. Our contributions are: (1) we identify and formalize two representational failure modes of single-frame VLAs in dynamic settings, *motion ambiguity* and *state aliasing*, and show that neither is a latency nor a capacity failure; (2) we address such failure modes and introduce two minimal plug-and-play inputs, TEMPO_{MOT}, an abstract past motion representation, and TEMPO_{ACT}, a compact proprioceptive history, that together make any pretrained VLA motion-aware without extensive pretraining efforts; and (3) we

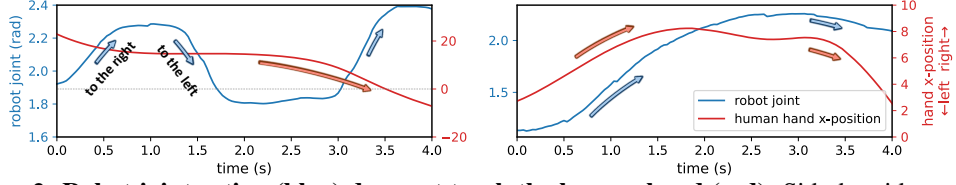


Figure 2: **Robot joint action (blue) does not track the human hand (red).** Side-by-side comparison of a Drop Catch rollout from the baseline (VLASH, left) and TEMPO (right). The baseline oscillates left-to-right continuously while the human hand moves in a single direction, whereas TEMPO closely tracks the hand.

release TEMPO-Bench, an object-motion benchmark of over 50k frames of human-robot dynamic manipulation with per-frame object-velocity annotations, together with a multiple-choice variant in the format of MVBench [6] and VLM4D [7] with object moving direction and motion magnitude question, targeted at evaluating motion understanding in VLMs and VLAs.

2 Related Work

Dynamic Tasks for Robotics. Dynamic manipulation has a long research history, with compelling results on table tennis [8, 9], object tossing [10, 11], and aerial catching [12, 13]. These systems achieve strong performance by explicitly modeling object physics and designing task-specific controllers, but each is an end-to-end pipeline for a single scenario that does not transfer to other tasks. In contrast, pretrained VLA models [14, 15, 3, 16, 17, 1, 2] generalize across tasks without per-task engineering; extending them to dynamic settings is the focus of this work.

Asynchronous Inference for Dynamic Tasks. A prominent line of work addresses the *execution timing* problem: inference latency of VLA models causes the executed chunk [18, 19] to be conditioned on a stale observation. SmolVLA [20], RTC [4], VLASH [5], and Leave No Observation Behind [21] each reduce or correct for this staleness. DynamicVLA [22] co-designs a compact VLA architecture with continuous inference and latent-aware streaming for lower-latency dynamic control. F2F-AP [23] takes a different route within the same problem: it predicts optical flow to synthesize an anticipated future frame, giving the policy visual context that compensates for latency. TEMPO is orthogonal to all of these: asynchronous inference addresses *when* a prediction is applied; TEMPO addresses *what* the policy is conditioned on. Even with a fresh or anticipated observation, a snapshot cannot encode how the scene is evolving or disambiguate sequential task phases. We build TEMPO on the asynchronous backbone of [5] so our experiments directly isolate the contribution of motion-aware representation; the two approaches are complementary and can be combined.

Visual and Visuomotor Representations in Robot Learning. A parallel line of work asks what *static* representation a policy should be conditioned on. Visual pretraining from large-scale ego-centric or robot video [24, 25, 26] learns transferable features [27] before any downstream policy fine-tuning. RoboAffordances [28] instead distills task-agnostic affordance priors from human videos, and HPT [29] scales pretraining across heterogeneous embodiments. UVT [30] and Anchor-Align [31] shape the training *signal* rather than the input. All of these methods condition the policy on a single frame. TEMPO instead supplying the temporal context that no single frame can carry.

3 Motion Ambiguity and State Aliasing

As briefed in Section 1, we observe two dominant failure modes recent VLA encounters when trained on dynamic tasks, both stemming from a Markovian assumption shared by every pretrained VLA today, where the current observation is treated as a sufficient context for inferring action. We illustrate each on our four dynamic tasks (setup in Section 5.1).

Motion ambiguity. A policy $\pi(o_t)$ conditioned on the current frame observes where objects are, but not where they are going. Recent works on dynamic tasks all focus on reducing inference latency,

which lets π act on a more recent observation, but does not inform the policy of object motion. Fig. 2 shows the resulting behavior on Drop Catch: when the policy is supposed to shadow human hand motion, the VLASH [5] policy instead produces actions that oscillate left-to-right. The same representational gap is also evident in our motion probe study in Section 6: linearly decoding object velocity from each baseline collapses on tasks where object motion is the only cue, with $R^2 \leq 0.03$ on Flick Catch and ≤ 0.09 on Wine Pour (Table 7). The policy has no basis for anticipating where a moving target will be.

State aliasing. A second failure mode arises from task structure and persists in the absence of motion. Multi-stage tasks force the policy to produce opposite actions from visually near-identical moments. For example, reaching for a plate at the start of a task and releasing it and pulling the gripper back look nearly the same from the head camera, but the corresponding future actions are opposite. The policy then hesitates between the two options: the gripper commits to neither, and the task fails to advance. Trained on the original Wine Pour and Flick Catch tasks, both asynchronous baselines fail every rollout (0/50; Table 1). To obtain nonzero baseline numbers for the main results in Table 4, we manually remove aliased state by trimming the release-and-retract phase from every training episode. Having to remove this segment at all is itself evidence of the failure for our baseline models.

What the two failures require. Both failures come down to the same gap: a single frame carries no temporal context. To perceive scene motion, the policy needs a signal built from recent observations; to tell task phases apart, it needs a signal built from its own recent actions. Both signals summarize data the policy is already receiving at inference time.

4 Temporal Encoding for Motion-aware Policy (TEMPO)

TEMPO supplies the two temporal signals that Section 3 identifies as missing: TEMPO_{MOT}, a motion encoder that summarizes recent scene dynamics, and TEMPO_{ACT}, a compact proprioceptive history. Both attach to a pretrained VLA with near-zero overhead (Fig. 3).

4.1 TEMPO_{MOT}: Captures Object and Scene Dynamics

TEMPO_{MOT} summarizes recent visual dynamics by cross-attending the current frame against a rolling cache of past frames (Fig. 3). Let $\mathbf{f}_\tau$ denote the patch features of observation o_τ , extracted by ϕ ’s per-frame encoder. Over a streaming window of length W , we maintain a cache $\mathcal{F}_t = \{\mathbf{f}_\tau\}_{\tau=t-W+1}^{t-1}$ of features from the preceding $W - 1$ frames, populated incrementally so that each $\mathbf{f}_\tau$ is computed exactly once and reused on subsequent steps at no additional cost. The motion code is then

$$\mathbf{m}_t = \phi(o_{t-W+1:t}) = \text{CrossAttn}(\mathbf{f}_t, \mathcal{F}_t), \quad (1)$$

with the current-frame features $\mathbf{f}_t$ as queries and historical visual features as keys and values.

We instantiate ϕ with a frozen pretrained video foundation model, used off-the-shelf without motion-specific fine-tuning; SAM 2.1-tiny [32] is our default. The choice is flexible: any video encoder whose features separate motion from appearance can serve as ϕ . Fig. A5 shows that object velocity is linearly decodable from $\mathbf{m}_t$ computed from different video encoders, substantially outperforming a position-only baseline, across several encoder families, and Table 2 confirms

Method	Flick Catch	Wine Pour
RTC [4]	0%	0%
VLASH [5]	0%	0%
TEMPO (Ours)	68%	97.6%

Table 1: **Success on the untrimmed Flick Catch and Wine Pour**, where each episode retains the release-and-retract phase that produces state aliasing. Both asynchronous baselines fail every rollout; TEMPO completes the task by using TEMPO_{ACT} to disambiguate phases the current frame cannot.

Motion encoder ϕ	Handover
SAM 2.1-Tiny [32] (default)	74%
Reducio [33]	76%
VidTwin [34]	60%
VideoLaVIT [35]	52%

Table 2: **Swapping the motion encoder** ϕ keeps Bottle Handover success comfortably above the VLASH baseline (38%).

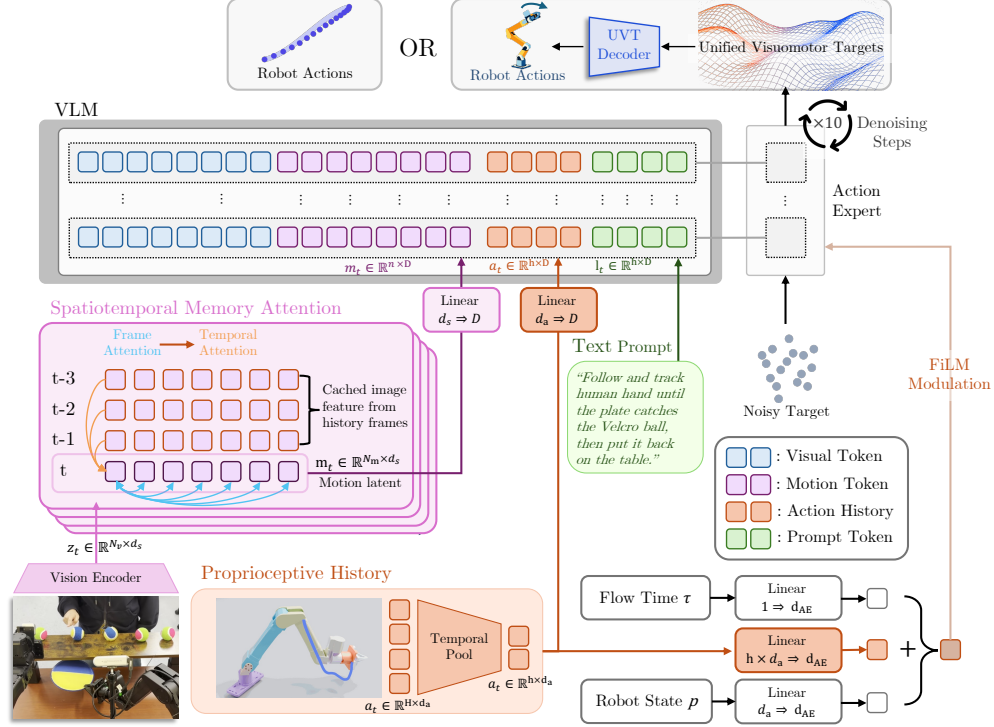


Figure 3: **TEMPO architecture.** TEMPO augments a pretrained VLM-based policy with two temporal inputs. TEMPO_{MOT} cross-attends the current frame’s patch features $\mathbf{f}_t$ against a streaming cache $\mathcal{F}_t$ of features from the preceding $W - 1$ frames, producing a motion code $\mathbf{m}_t$ that encodes scene dynamics. TEMPO_{ACT} mean pools the robot’s recent proprioceptive command history into K fixed-size buckets, yielding a compact summary $\mathbf{c}_t$. Both signals attach to the pretrained backbone with near-zero overhead: $\mathbf{m}_t$ and $\mathbf{c}_t$ are projected to the VLM’s token dimension and appended to its prefix, and $\mathbf{c}_t$ additionally conditions the action expert via an AdaRMS residual.

the same conclusion at the task level: swapping ϕ

across four video foundation models keeps Bottle Handover success in the 52–76% range, all well above the VLASH baseline. Reducio slightly exceeds our SAM default; we still adopt SAM 2.1-Tiny for the streaming speed required at real-time control rates (Tab. 3).

4.2 TEMPO_{ACT}: Resolves State Aliasing

Where TEMPO_{MOT} captures how the scene is moving, TEMPO_{ACT} captures what the robot has been doing. A compact summary of recent proprioceptive history gives the temporal context for the robot policy to commit to the correct task phase. Without resolving state aliasing, the policy averages over multiple valid actions for the same observation, often producing indecisive behavior (e.g., pausing) or oscillating between conflicting actions, as seen in our baseline rollouts (Fig. A1).

Raw proprioceptive history is long, high-frequency, and largely redundant, so we compact it into a fixed-size summary $\mathbf{c}_t$ (TEMPO_{ACT}). Let the proprioceptive command at time τ be $q_\tau \in \mathbb{R}^{d_q}$. We consider the most recent H commands preceding time t , partition them into K contiguous temporal buckets $B_1, \dots, B_K$ of equal length L (so that $H = KL$), ordered from oldest to newest, and summarize each bucket by its mean:

$$\mathbf{c}_t = (\bar{q}_t^{(1)}, \dots, \bar{q}_t^{(K)}), \quad \bar{q}_t^{(k)} = \frac{1}{L} \sum_{\tau \in B_k} q_\tau \in \mathbb{R}^{d_q}, \quad q_\tau := 0 \text{ for } \tau < \text{episode start.} \quad (2)$$

We use $K = 10$ buckets, each averaging $L = 30$ consecutive commands. Buckets that fall entirely before the episode start are marked with a pad flag. The resulting representation $\mathbf{c}_t \in \mathbb{R}^{K \times d_q}$ always

contains K tokens, regardless of how much history it summarizes. Extending the temporal horizon only changes the bucket length L , not the number of tokens. This simple, parameter-free aggregation preserves a coarse trend of robot’s recent motion while adding negligible computational overhead.

4.3 Plug-and-Play Temporal Integration

We add the two signals to a pretrained VLA while keeping the pretrained backbone largely intact (Fig. 3). We build on VLASH’s asynchronous-inference pipeline [5] as the backbone, though our additions are agnostic to this choice. $\mathbf{m}_t$ is projected to the VLM’s token dimension and appended to its prefix alongside the current multi-view observation and the language instruction. The action expert reads the resulting VLM prefix through the cross-attention layers.

The K bucket tokens of $\mathbf{c}_t$ are used in two ways. First, they are projected into VLM’s prefix tokens dimension and appended to the VLM token stream, allowing the multimodal prefix to include the robot’s recent proprioceptive history. Second, the same bucket sequence is flattened and passed through a conditioning MLP to produce an AdaRMS residual for the action expert, modulating the denoising process directly.

Together, the two temporal signals provide complementary temporal contexts to the VLA. The added conditioning path is zero-initialized, so the pretrained policy’s behavior is preserved at the start of fine-tuning. Flow-matching loss is unchanged, and TEMPO adds only 2M (0.08%) parameters.

4.4 Real-Time Deployment via Parallel Encoding

TEMPO_{MOT} is encoded in a dedicated background thread, decoupled from the control loop: the motion encoder consumes camera frames at sensor rate (~ 11 ms per frame on an NVIDIA RTX PRO 6000) and continuously updates the latest motion token, which the policy reads at each control step. The only overhead the policy pays is about 1.5 ms for attending to the extra token (35.3 ms vs. 33.8 ms median forward time; Tab. 3), versus approximately 128 ms per chunk for the asynchronous baseline RTC [4]. Because ϕ operates over a fixed-size streaming window, its per-frame cost stays constant regardless of history length, so TEMPO adds motion awareness without impacting the policy’s control rate.

Method	Median
VLASH [5]	33.8 ms
RTC [4]	127.7 ms
TEMPO (Ours)	35.3 ms

Table 3: Inference latency comparison across methods.

5 Experiments

Method	Bottle Handover	Drop Catch	Flick Catch	Wine Pour	Average
RTC [4]	44%	80%	48%	80.8%	63.2%
VLASH [5]	38%	38%	20%	94.0%	47.5%
TEMPO (Ours)	74%	66%	66%	98.1%	76.0%
TEMPO+UVT [30]	88%	88%	76%	96.7%	87.2%

Table 4: **Success rates across the four dynamic tasks.** Each entry is the percentage of 50 real-world rollouts in which the policy completes the task, evaluated under varied object speeds, appearances, and approach directions. For Wine Pour task, instead of recording binary success or failure, we measure the percentage of the ”wine” mass retained in the pot at the end of the pour.

5.1 Experimental Setup

We design four tasks to expose the two ambiguities identified in Section 1: motion ambiguity and state aliasing. We ask whether TEMPO resolves each kind of ambiguity through TEMPO_{MOT} and TEMPO_{ACT}. We compare TEMPO against two state-of-the-art asynchronous baselines designed for responsive task execution, RTC [4] and VLASH [5]. For Wine Pour and Flick Catch, both baselines collapse to 0% on the full task because the release-and-retract phase is perceptually aliased with

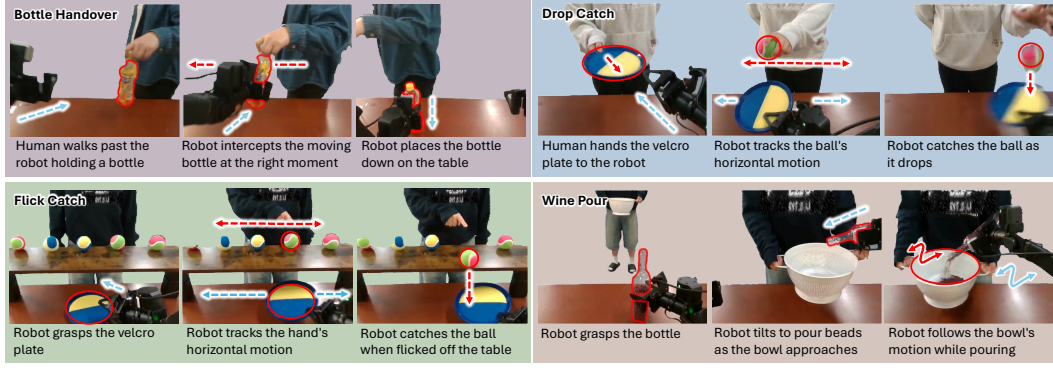


Figure 4: **Task Setup.** We benchmark all models across four comprehensive dynamic tasks.

reaching. As a result, a baseline that sees the gripper near the bottle or plate cannot tell whether to grasp or withdraw and oscillates between the two (more detailed visualization and analysis in Section A). To avoid a trivial comparison, we report quantitative results on a trimmed version that removes the release-and-retract phase, eliminating this artifact. We emphasize that TEMPO disambiguates these phases through $\text{TEMPO}_{\text{MOT}}$ and $\text{TEMPO}_{\text{ACT}}$, so the gain in Table 4 is a lower bound on TEMPO’s full-task advantage. For qualitative rollouts of the baselines’ complete failure and TEMPO’s behavior, please see our project webpage and Section A.

Task properties. Each of the four tasks is dynamic and shares four properties that a current-frame observation alone cannot resolve. **(P1)** The target object is moving at varying motion. Therefore, reacting based on the current snapshot alone is inaccurate in both position and timing. **(P2)** The relevant target shifts during the episode. As a result, the policy must redirect its attention to whichever object currently matters. **(P3)** Each task decomposes into a sequence of subtasks (approach, grasp, track, release), and the correct next action depends on which subtask has already been completed. **(P4)** The policy faces state ambiguity. From the current frame alone, multiple next actions are plausible, and a perceptually identical state may demand opposite actions depending on what has come before.

Bottle Handover (P1). The robot intercepts a bottle from a human walking by from either direction. We vary the walking speed, bottle’s height, and holding pose across episodes, so the policy must estimate the bottle’s velocity to time and place the grasp correctly.

Drop Catch (P1, P3). The robot catches a ball dropped by a horizontally moving human hand. Each episode begins with the human handing a Velcro plate to the robot, after which the robot tracks the human’s hand laterally before catching the released ball. This task includes a moving dynamic object (P1) and multiple sequential subtasks (P3).

Flick Catch (P1–P4). Several Velcro balls rest on a long horizontal platform. The robot first picks up a Velcro plate from the table (P3). A human hand then hovers over the balls and, at a random time, flicks one off the platform. The robot must first track the hand, then switch to tracking the released ball to catch it in time (P2, P1). Because the robot picks up and later places down the plate at the same location, the pick-up is perceptually aliased with the subsequent drop-off (P4), making this the most challenging of the four tasks.

Wine Pour (P1–P4). The robot pours beads, an alternative to wine, into a pot that the human moves around. The task decomposes into three subtasks (P3): pick up the bottle from the table, pour while tracking the pot movement, and put the bottle back down. Pick-up and put-down produce visually similar states yet require opposite actions, producing state aliasing (P4). Once pouring begins, the target shifts from the bottle to the pot (P2), whose motion must be tracked (P1). Since every rollout inevitably spilled some beads, this task’s success is determined by the percentage of beads’ mass retained in the pot at the end of the pour.

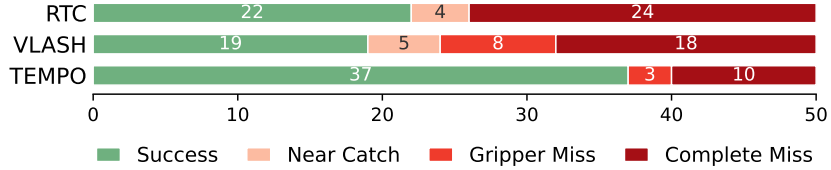


Figure 5: **Failure-mode breakdown on Bottle Handover**, we categorized and visually examined each episode in deployment rollouts and attributed each model’s failure to different categories.

5.2 Experimental Results

Success Rate on Dynamic Tasks. As shown in Table 4, we extensively compare our method against each of the SOTA asynchronous VLA models. For each task, we calculated success rate for each model based on 50 trials following the evaluation protocol established under Section 5.1. On simpler tasks like Drop Catch, TEMPO achieves a competitive success rate compared to RTC, while VLASH frequently overshoots the human motion and misses the dropped ball. Across the remaining tasks where the object motion is more uncertain or the object of interest shifts, TEMPO consistently outperforms both baselines, including +36% on Bottle Handover, +4.1% on Wine Pour, and +46% on Flick Catch. We observed closer tracking of the human hand by the robot gripper in Drop Catch and Flick Catch, as well as more precisely timed grasps in Bottle Handover.

Failure mode analysis. A single success rate hides *why* a policy fails, so we visually examined all 50 rollouts of every model on Bottle Handover and labeled each as a success or one of three failure modes (Fig. 5). The dominant failure for both baselines is *Complete Miss*, where the arm reaches toward the wrong place and misses the bottle entirely, indicating that the policy mispredicts where the moving bottle will be. VLASH and RTC commit 18 and 24 Complete Misses respectively, while the timing-related *Gripper Miss* and the near-success *Near Catch* are comparatively rare. By conditioning on recent motion, TEMPO cuts Complete Misses to 10 and raises success from 19 (VLASH) and 22 (RTC) to 37 out of 50. Although both asynchronous pipelines significantly reduce inference delay, a lack of motion understanding still hurts dynamic-task performance.

Plug-and-play across VLA backbones. The two temporal inputs of TEMPO are not backbone-specific. We attach the same TEMPO_{MOT} and TEMPO_{ACT} to two additional pretrained VLAs, $\pi_{0.5}$ [2] (non-asynchronous) and RTC [4], and retrain each on the same Bottle Handover demos. Every backbone we tried improves substantially: $\pi_{0.5}$ from 26% to 56%, RTC from 44% to 80%, and VLASH from 38% to 74% (Table 5). TEMPO helps both asynchronous and non-asynchronous backbones, indicating that motion-aware perception is a general upgrade rather than a VLASH-specific effect.

VLA backbone	Base	+ TEMPO
$\pi_{0.5}$ [2]	26%	56%
RTC [4]	44%	80%
VLASH [5]	38%	74%

Table 5: **TEMPO is plug-and-play across VLA backbones.** Adding TEMPO_{MOT} and TEMPO_{ACT} improves Bottle Handover on all backbones, including the non-asynchronous $\pi_{0.5}$.

Extending TEMPO with a compact training target. With TEMPO closing the input-side gap, a natural line of question being whether anything remains on the output side. Our prior work, Unified Visuomotor Target (UVT) [30], argues that it does: VLAs are typically trained to predict raw joint-position chunks, which are high-dimensional and jitter-prone, and swapping this target for a compact motion primitive aligned with object motion tightens the target the policy has to learn. Since TEMPO and UVT act on different sides of the policy, they combine without redundancy: TEMPO+UVT achieves the highest overall success rate in Table 4 and outperforms baselines on 3 of the 4 tasks, further pushing dynamic task performances.

Task	RTC [4]	VLASH [5]	TEMPO
Bottle Handover	0.46	0.43	0.52
Drop Catch	0.31	0.28	0.33
Flick Catch	−0.07	0.03	0.57
Wine Pour	0.06	0.09	0.44
Average	0.19	0.21	0.47

Table 7: **Motion probing of the trained VLA hidden state.** MLP probe on object velocity across our dynamic tasks (R^2). On Flick Catch and Wine Pour, where motion is the only usable cue, the asynchronous baselines’ representations collapse to near zero while TEMPO’s remains informative.

6 Analysis and Ablations

Having shown TEMPO outperforms recent VLA baselines on dynamic tasks, we now turn to *why*. We evaluate whether the policy relies on TEMPO_{MOT} and TEMPO_{ACT} at the task stages where their corresponding ambiguities arise, using single-signal ablations and hidden-state probing.

Each temporal signal is individually necessary. To verify that both TEMPO_{MOT} and TEMPO_{ACT} carry non-redundant information for task success, we retrain TEMPO with each single signal removed and evaluate on Bottle Handover (Table 6). Removing either signal substantially reduces success, and both single-signal variants still outperform the VLASH baseline (38%), confirming that each temporal input is independently useful. Combining the two lifts success to 74%, above what either signal alone achieves. Each signal is also consulted at a distinct moment in the episode; see Section D for the per-frame ablation MSE.

Configuration	Handover
TEMPO _{ACT} only	52%
TEMPO _{MOT} only	56%
TEMPO (full)	74%

Table 6: **Both temporal signals contribute to the task.** Removing either TEMPO_{MOT} or TEMPO_{ACT} reduces Bottle Handover success; the two together outperform either alone.

Baseline hidden states lack motion on the hardest tasks. We test whether the trained policy encodes object motion by probing its hidden state at a fixed token position, following the same protocol as prior VLM probe works [36, 37]. For each task we annotate the per-frame position of the object of interest and compute its pixel-space velocity, yielding an object-motion benchmark of over 50k annotated frames of human-robot dynamic manipulation that we release as TEMPO-Bench, together with a multiple-choice variant in the format of MVBench [6] and VLM4D [7]. Here we report the regression setting: we fit an MLP probe from each model’s hidden state to the velocity and report R^2 (Table 7).

TEMPO produces the strongest probe result on every row of Table 7. Its hidden state decodes object velocity substantially better than the asynchronous baselines on the dynamic tasks where motion is the only cue (Flick Catch $R^2 = 0.57$ vs. ≤ 0.03 ; Wine Pour $R^2 = 0.44$ vs. ≤ 0.09), where the baseline probes collapse to near zero. Since RTC, VLASH, and TEMPO share the same $\pi_{0.5}$ backbone and pretraining, TEMPO’s gain over the two is attributed to its motion-conditioned fine-tuning: TEMPO_{MOT} and TEMPO_{ACT} leave a linearly-decodable motion signal in the hidden state that the other $\pi_{0.5}$ -family backbones do not carry. We additionally ablate the choice of motion encoder, probing several video foundation models’ latents for object velocity, in Section E.

7 Conclusion

The gap between pretrained VLAs and dynamic manipulation is primarily representational: motion ambiguity and state aliasing each have a fix from representations already in wide use, and our failure mode breakdown (Fig. 5) and ablation analyses (Sec. 6) confirm that the two failures are separable and that each signal addresses its own. TEMPO implements this as two plug-and-play inputs with near-zero overhead; the gains on the untrimmed Wine Pour and Flick Catch, where asynchronous baselines score zero (Table 1), show that timing is not the bottleneck.

Acknowledgments

We thank CoRL reviewers and AC for their valuable feedback on improving papers, along with Basavasagar Patil for his careful reviews of the final draft of our paper.

References

- [1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [2] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024.
- [4] K. Black, M. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026.
- [5] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han. Vlash: Real-time vlas via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.
- [6] K. Li, Y. Wang, Y. He, Y. Li, Y. Wang, Y. Liu, Z. Wang, J. Xu, G. Chen, P. Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22195–22206, 2024.
- [7] S. Zhou, A. Vilesov, X. He, Z. Wan, S. Zhang, A. Nagachandra, D. Chang, D. Chen, X. E. Wang, and A. Kadambi. Vlm4d: Towards spatiotemporal awareness in vision language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8600–8612, 2025.
- [8] D. B. D’Ambrosio, J. Abelian, S. Abeyruwan, M. Ahn, A. Bewley, J. Boyd, K. Choromanski, O. Cortes, E. Coumans, T. Ding, et al. Robotic table tennis: A case study into a high speed learning system. In *Robotics: Science and Systems*, 2023.
- [9] D. B. D’Ambrosio, S. Abeyruwan, L. Graesser, A. Iscen, H. B. Amor, A. Bewley, B. J. Reed, K. Reymann, L. Takayama, Y. Tassa, et al. Achieving human level competitive robot table tennis. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 74–82. IEEE, 2025.
- [10] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser. Tossingbot: Learning to throw arbitrary objects with residual physics. *IEEE Transactions on Robotics*, 36(4):1307–1319, 2020.
- [11] H. Ha and S. Song. Flingbot: The unreasonable effectiveness of dynamic manipulation for cloth unfolding. In *Conference on Robot Learning*, pages 24–33. PMLR, 2022.
- [12] Y. Zhang, T. Liang, Z. Chen, Y. Ze, and H. Xu. Catch it! learning to catch in flight with mobile dexterous hands. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14385–14391. IEEE, 2025.

- [13] B. Huang, Y. Chen, T. Wang, Y. Qin, Y. Yang, N. Atanasov, and X. Wang. Dynamic handover: Throw and catch with bimanual hands. In *Conference on Robot Learning*, 2023.
- [14] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023.
- [15] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. In *International Conference on Machine Learning*, 2023.
- [16] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [17] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems*, 2024.
- [18] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [19] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.
- [20] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [21] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224*, 2025.
- [22] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu. Dynamicvla: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026.
- [23] H. Wei, X. Xu, Z. Cheng, H. Yin, A. Ma, B. Yu, J. Zhou, and J. Lu. F2f-ap: Flow-to-future asynchronous policy for real-time dynamic manipulation. *arXiv preprint arXiv:2604.02408*, 2026.
- [24] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation. In *Conference on Robot Learning*. PMLR, 2022.
- [25] I. Radosavovic, T. Xiao, S. James, P. Abbeel, J. Malik, and T. Darrell. Real-world robot learning with masked visual pre-training. In *Conference on Robot Learning*, pages 416–426. PMLR, 2022.
- [26] I. Radosavovic, B. Shi, L. Fu, K. Goldberg, T. Darrell, and J. Malik. Robot learning with sensorimotor pre-training. In *Conference on Robot Learning*, pages 683–693. PMLR, 2023.
- [27] S. Dasari, M. K. Srirama, U. Jain, and A. Gupta. An unbiased look at datasets for visuo-motor pre-training. In *Conference on Robot Learning*, 2023.
- [28] S. Bahl, R. Mendonca, L. Chen, U. Jain, and D. Pathak. Affordances from human videos as a versatile representation for robotics. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [29] L. Wang, X. Chen, J. Zhao, and K. He. Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2024.

- [30] Z. Feng and U. Jain. Unified visuomotor targets: Supervising vlas beyond physical actions. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026.
- [31] D. Dalal, S. Patel, C. Jain, J. Kim, U. Mishra, A. Baratian, H. Ha, H. Ji, S. Lazebnik, and U. Jain. Generalizable vla finetuning via representation anchoring and language-action alignment. *arXiv preprint arXiv:2607.13429*, 2026.
- [32] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, et al. Sam 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025.
- [33] R. Tian, Q. Dai, J. Bao, K. Qiu, Y. Yang, C. Luo, Z. Wu, and Y.-G. Jiang. Reducio! generating 1k video within 16 seconds using extremely compressed motion latents. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19237–19247, 2025.
- [34] Y. Wang, J. Guo, X. Xie, T. He, X. Sun, and J. Bian. VidtwiN: Video vae with decoupled structure and dynamics. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 22922–22932, 2025.
- [35] Y. Jin, Z. Sun, K. Xu, L. Chen, H. Jiang, Q. Huang, C. Song, Y. Liu, D. Zhang, Y. Song, et al. Video-lavit: Unified video-language pre-training with decoupled visual-motional tokenization. In *International Conference on Machine Learning*, 2024.
- [36] Q. Zhao, M. Xu, K. Gupta, A. Asthana, L. Zheng, and S. Gould. The first to know: How token distributions reveal hidden knowledge in large vision-language models? In *European Conference on Computer Vision*, pages 127–142. Springer, 2024.
- [37] H. Lu, H. Li, P. S. Shahani, S. Herbers, and M. Scheutz. Probing a vision-language-action model for symbolic states and integration into a cognitive architecture. In *2025 IEEE International Conference on AI and Data Analytics (ICAD)*, pages 1–8. IEEE, 2025.
- [38] I2RT Robotics. Yam ultra 6-dof robotic arm, 2024.

Appendix

In this Appendix, we include extensions of analysis introduced in the main paper and additional technical details of our method, setup, and evaluation protocol. Qualitative rollouts, failure mode analysis, and more details are available on our project page: <https://tempo-robot.github.io/>.

- **Section A, Additional Failure Mode Analysis.** Supplements the trimming discussion in Section 5.1 and the success rates in Table 4. Explains why baselines collapse to 0% on the original Flick Catch and Wine Pour, and reports TEMPO’s performance on the untrimmed tasks (Table 1, Fig. A1).
- **Section B, Experimental Setup Details.** Supplements Section 5.1. Lists the hardware platform, evaluation protocol, and training details for each method.
- **Section C, Per-Task Failure Mode Breakdown.** Supplements Section 5.2 and the Bottle Handover breakdown in Fig. 5. Extends the same per-rollout categorization to Flick Catch (Fig. A2) and Drop Catch (Fig. A3).
- **Section D, Input Ablation.** Supplements the single-signal ablations in Section 6. Measures per-frame causal reliance on TEMPO_{MOT} and TEMPO_{ACT} via input ablation MSE (Fig. A4).
- **Section E, Motion Encoder Selection.** Supplements the motion-encoder definition in Section 4 (ϕ in TEMPO_{MOT}). Justifies our choice of SAM 2.1-Tiny by linearly probing several candidate encoders for object velocity (Fig. A5).

A Additional Failure Mode Analysis

Supplements the trimming discussion in Section 5.1 and Table 4; extends the state-aliasing evidence in Section 3. In our initial setup, every method was trained and evaluated on the complete, untrimmed versions of all four tasks. On Flick Catch and Wine Pour, both asynchronous baselines failed every rollout. The cause is state aliasing rather than reaction speed or accuracy: each episode contains a reach-out phase, in which the gripper approaches the velcro plate or the bottle, and a release-and-retract phase, in which the gripper recedes from the same object after grasping or pouring. The two phases produce visually near-identical observations yet demand opposite actions, and a single-frame policy has no cue to disambiguate them, so the gripper oscillates between extending toward and withdrawing from the object and never commits long enough to make progress (Fig. A1; see qualitative rollouts on our project webpage).

Because both baselines collapse to 0% on the untrimmed task, a direct comparison there is uninformative. To enable a non-trivial comparison that isolates motion ambiguity from state aliasing, we manually trim the release-and-retract segment from every episode of the Flick Catch and Wine Pour training and evaluation data; the success rates on this trimmed version are reported in Table 4. The trimmed task still requires motion-aware perception (target tracking and timing) but removes the state-aliased segment.

To verify that TEMPO resolves state aliasing rather than merely benefiting from this trimming, we additionally train and evaluate every method on the original, untrimmed episodes. Table 1 (in Section 3) reports the resulting success rates: RTC and VLASH again fail every rollout, while TEMPO succeeds at 68% on Flick Catch and 97.6% on Wine Pour. We attribute the gain to TEMPO_{ACT}: the proprioceptive history distinguishes task phases that the current frame cannot, letting the policy commit to the correct action rather than hedging between reach-out and retract.

B Experimental Setup Details

Supplements Section 5.1.

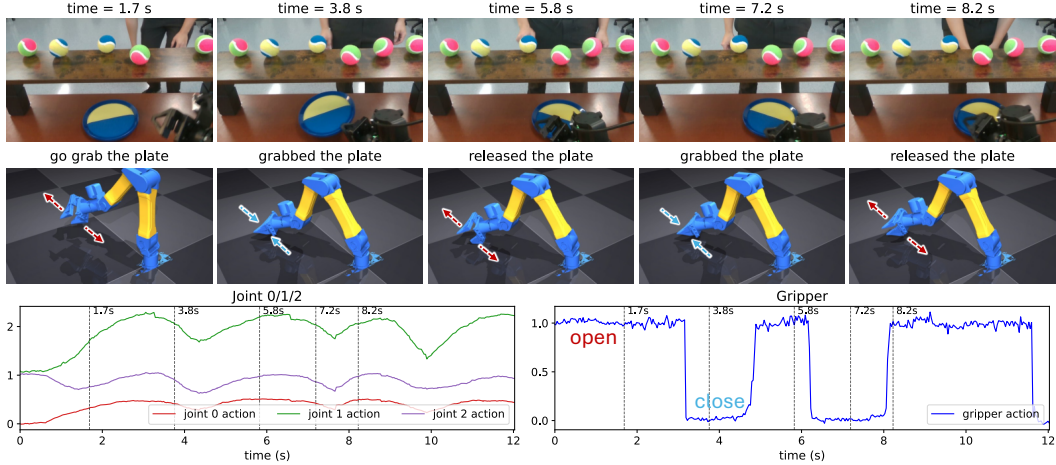


Figure A1: **VLASH rollout on the untrimmed Flick Catch task.** The task is to grip the velcro plate and follow the human hand. VLASH’s gripper hovers near the plate without committing to grasp or retract, because a single-frame observation cannot differentiate reaching for the plate from releasing it. RTC exhibits the same failure (see project page).

B.1 Hardware Platform

All experiments are conducted on a bimanual workstation consisting of two I2RT YAM Ultra robot arms [38], each with 7 degrees of freedom. The two arms are mounted and clamped to the table edge with their bases separated by 24 inches (61 cm). Both arms are placed on a rigid table measuring 6 ft $\times$ 29.5 in $\times$ 28 in (width $\times$ depth $\times$ height; 183 $\times$ 75 $\times$ 71 cm). Each arm is equipped with a parallel-jaw gripper.

Visual observations are provided by three cameras: one fixed third-person camera positioned between the two arms and two wrist-mounted cameras. Each camera streams 224p at 30 Hz. Policies are executed at 30 Hz, if possible.

B.2 Evaluation Protocol

For the Drop Catch task, a single ball is released above the workspace from a height of 13–18 in (33–46 cm), measured from the table surface to the release point. The ball is tennis-ball sized and covered in a velcro-compatible material so that it adheres to the velcro plate that the gripper holds.

For the Flick Catch task, 5–6 balls are arranged on a shelf measuring 31.5 $\times$ 7.9 $\times$ 8.5 in (80 $\times$ 20 $\times$ 21.6 cm; width $\times$ depth $\times$ height), positioned near the table edge opposite to where the arms are clamped. These balls are identical to the one used in the Drop Catch task.

To ensure a fair evaluation, the human operator releasing the ball performs each trial with eyes closed while wearing headphones, preventing them from anticipating the robot’s position and biasing the release point toward it.

B.3 Training Details

Each policy is trained on eight NVIDIA RTX PRO 6000 Blackwell GPUs. Training a single policy takes approximately 2 hours. More specifically, with an effective batch size of 128 the per-iteration time is 0.49s for VLASH, 0.46s for RTC, and 0.50s for TEMPO. RTC does not have an openly available codebase, so we reimplement it faithfully from its paper.

C Per-Task Failure Mode Breakdown

Supplements Section 5.2 and Fig. 5. To complement the Bottle Handover analysis in Section 5.2 (Fig. 5), we apply the same protocol to Flick Catch and Drop Catch: we visually examine all 50 rollouts per method on the trimmed task and assign each to a success or to one failure category. The categories differ across tasks because each task has different failure modes.

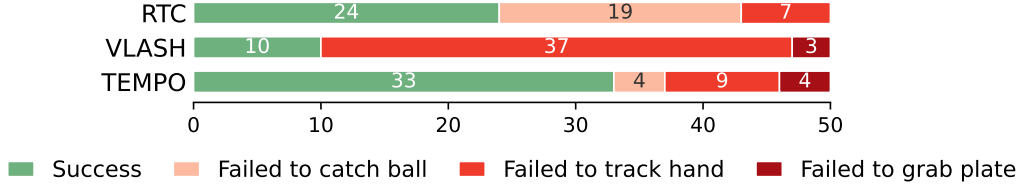


Figure A2: **Failure-mode breakdown on Flick Catch.** Each of the 50 rollouts is decomposed into success plus three failure modes corresponding to the three task stages (grab plate, track hand, catch ball). VLASH’s failures concentrate in hand-tracking and RTC struggles most at tracking the ball’s trajectory (see project page); TEMPO cuts both failure modes.

Flick Catch (Fig. A2). We assign each failed rollout to one of three failure modes corresponding to the three task stages: grabbing the plate, tracking the hand, or catching the flicked ball. VLASH’s failures concentrate almost entirely in hand-tracking: in 37 of 50 trials the policy fails to follow the human hand and therefore never reaches the catching stage. RTC reacts to the hand but loses the flicked ball, with ball-catch failures accounting for 19 trials, the dominant mode for RTC. TEMPO cuts VLASH’s hand-tracking failures from 37 to 9 and RTC’s ball-catch failures from 19 to 4; the residual failures are spread across all three stages rather than concentrated in any single one.

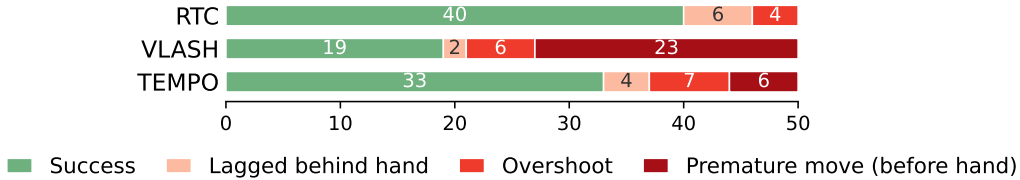


Figure A3: **Failure-mode breakdown on Drop Catch.** RTC scores highest as Drop Catch is dominated by reactive timing rather than motion or state ambiguity. VLASH’s dominant failure is premature movement before the human’s hand begins to move; TEMPO cuts this failure by nearly 4×.

Drop Catch (Fig. A3). Drop Catch is the timing-dominated task on which RTC scores highest (Table 4); its only failures are a few overshoots (4) and lags (6). VLASH, despite also being asynchronous, exhibits a markedly different profile: it *moves before the human’s hand* in 23 of 50 trials, a near-majority failure that pulls its success rate down to 19. TEMPO cuts this premature-move failure by nearly 4× (from 23 to 6), with only a small change in overshoots (6 → 7). The premature-move reduction is consistent with the role of TEMPO_{ACT}: the proprioceptive history records what the policy has already begun, so it does not re-initiate a move on every frame.

D Input Ablation

Supplements the modality-ablation analysis in Section 6. To test whether TEMPO_{MOT} and TEMPO_{ACT} are both necessary and contribute meaningfully to task performance, we run a per-frame ablation on a representative episode of the Flick Catch task (Fig. A4). For each frame we “remove” each input modality by freezing TEMPO_{MOT} or TEMPO_{ACT} to its first-frame value and

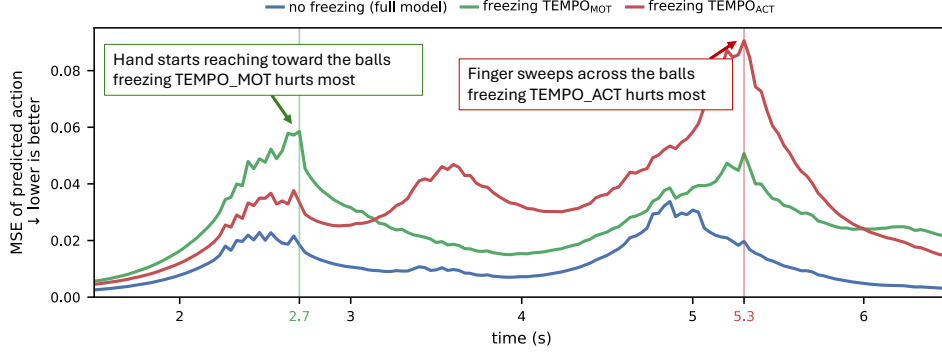
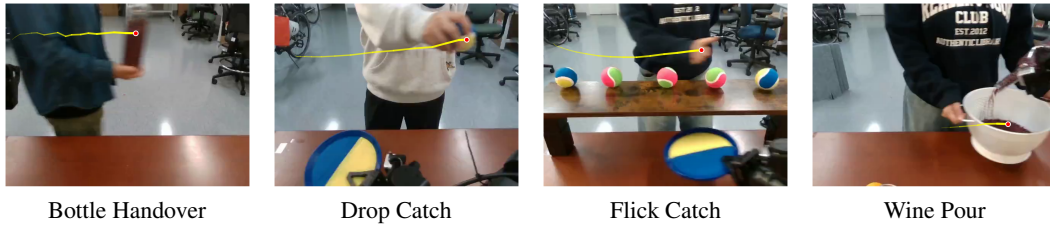


Figure A4: **Both $\text{TEMPO}_{\text{MOT}}$ and $\text{TEMPO}_{\text{ACT}}$ are necessary.** Per-frame action MSE for the full TEMPO model (blue) vs. the two single-signal ablations, evaluated on the Flick Catch task. Freezing $\text{TEMPO}_{\text{MOT}}$ to its first-frame value (green) hurts most at 2.7 s, the beginning of the task where human hand starts to move. Freezing $\text{TEMPO}_{\text{ACT}}$ (red) hurts most at 5.3 s, the middle of the task where the finger sweeps horizontally across the balls before picking one and flicking it. Thus, each signal plays an important role at a different moment in the task.

measure the resulting predicted-action MSE; curves are EMA-smoothed for readability. Freezing either modality substantially degrades the policy’s action prediction at different points in the episode. Freezing $\text{TEMPO}_{\text{MOT}}$ hurts most at 2.7 s, the beginning of the task where large motions occur: MSE rises from 0.019 to 0.059 ($3.1\times$), while freezing $\text{TEMPO}_{\text{ACT}}$ costs only 0.033. Freezing $\text{TEMPO}_{\text{ACT}}$ hurts most at 5.3 s, the middle of the task where the finger sweeps horizontally across the balls before picking one and flicking it, and there the pattern reverses: freezing $\text{TEMPO}_{\text{ACT}}$ hurts more than $\text{TEMPO}_{\text{MOT}}$ (0.091 vs. 0.051; full 0.020). Each signal helps a different phase of the task.

E Motion Encoder Selection



Initialization for $\text{TEMPO}_{\text{MOT}}$	Bottle Handover	Drop Catch	Flick Catch	Wine Pour	Average
Object Position	0.24	0.03	0.03	0.02	0.08
VideoLaVIT [35]	0.45	0.25	0.28	0.36	0.34
Reducio [33]	0.50	0.36	0.38	0.32	0.39
SAM 2 [32]	0.58	0.37	0.50	0.39	0.46
VidTwin [34]	0.59	0.44	0.43	0.45	0.48

Figure A5: **Linear probe of candidate initializations for $\text{TEMPO}_{\text{MOT}}$.** *Top*: one example frame per task showing the object of interest (red dot) and its recent trajectory (yellow line); the pixel-space velocity of this object is the regression target the linear probe must predict. *Bottom*: per-task velocity R^2 for each candidate initialization, along with the average. Higher is better; the raw object-position baseline ($R^2 = 0.08$ on average) cannot recover velocity from a single position, while every learned motion encoder does so substantially better.

Supplements the motion-encoder definition in Section 4 (the ϕ in $\text{TEMPO}_{\text{MOT}}$). TEMPO is agnostic to the choice of motion encoder; here we justify ours. Reusing the per-frame object-velocity annotation from our motion probe (Sec. 6; the per-task probing target is visualized at the top of Fig. A5), we fit a *linear* probe from each candidate motion latent to the object’s velocity: if the latent cap-

tures the object’s motion, a linear readout should recover the velocity with high R^2 . Every learned encoder decodes velocity far above the raw object position (avg. $R^2 = 0.08$), confirming that object motion is genuinely present in the latent and that a single position, like a single frame, does not by itself encode meaningful information to infer object motion. SAM 2 and VidTwin score highest; we adopt SAM 2.1-Tiny because it pairs near-top motion correlation with the streaming speed required for real-time deployment (Table 3).